Detection of Unknown Attacks Through Encrypted Traffic: A Gaussian Prototype-Aided Variational Autoencoder Framework

Qianwei Meng, Jing Tao, Qingjun Yuan, Guangsong Li, Yongjuan Wang, Bing Gao, and Siqi Lu

***Abstract*—The identification of encrypted network traffic presents a pivotal challenge in detecting unknown malicious traffic. Unlike closed-set identification, which primarily classifies known traffic classes, detecting unknown malicious traffic necessitates both accurate classification of known traffic and the identification of previously unseen traffic classes. Existing methods often face difficulties in effectively constraining the distribution size of known classes in the representation space and frequently misclassifying unknown classes as known. To address these challenges, we propose Open-Detect, a robust theoretical framework for detecting unknown malicious traffic, which leverages advanced deep learning techniques, such as variational autoencoders and Gaussian prototypes. Open-Detect introduces two primary constraints: a generative constraint, which enhances intra-class compactness, and a discriminative constraint, which optimizes inter-class separation. These constraints collectively mitigate the risks of misclassifying known classes and failing to detect unknown classes. In Open-Detect, network flows are transformed into grayscale images, and each known traffic class is mapped to a unique Gaussian prototype in the latent space. This design ensures tight clustering of samples within the same class and clear separation of samples between different classes. The detection of unknown malicious traffic is performed based on the distance between samples and these prototypes. Extensive experiments conducted on multiple publicly available datasets substantiate the efficacy of Open-Detect. The results reveal significant improvements in intra-class compactness and inter-class separation, enabling superior performance in both closed-world and open-world scenarios, particularly for detecting unknown malicious traffic. Our code is available at: https://github.com/niebikong/Open-Detect**

Received 9 December 2024; revised 22 May 2025, 30 July 2025, and 1 September 2025; accepted 15 September 2025. Date of publication 19 September 2025; date of current version 14 October 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB2705000, in part by the National Natural Science Foundation of China under Grant 62276091, and in part by the Natural Science Foundation of Henan under Grant 252300420992. The associate editor coordinating the review of this article and approving it for publication was Dr. Ming Li. *(Corresponding author: Qingjun Yuan.)*

Qianwei Meng, Guangsong Li, Yongjuan Wang, Bing Gao, and Siqi Lu are with Henan Key Laboratory of Network Cryptography Technology and the Key Laboratory of Cyberspace Security, Ministry of Education, Information Engineering University, Zhengzhou 450001, China.

Jing Tao is with the MoE Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: jtao@mail.xjtu.edu.cn).

Qingjun Yuan is with the MoE Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an 710049, China, and also with Henan Key Laboratory of Network Cryptography Technology and the Key Laboratory of Cyberspace Security, Ministry of Education, Information Engineering University, Zhengzhou 450001, China (e-mail: gcxyuan@outlook.com).

Digital Object Identifier 10.1109/TIFS.2025.3612141

***Index Terms*—Intrusion detection system, unknown attack, encrypted traffic.**

I. Introduction

With the rapid advancements in deep learning technologies, encrypted traffic classification methods have achieved remarkable progress in applications such as application classification, website fingerprinting, and encrypted video content recognition [1], [2], [3], [4]. Despite the successes of deep learning in these areas, several significant challenges persist. One critical issue lies in the dynamic and evolving nature of network environments. Traditional intrusion detection systems often rely on closed-set classification models that are ill-equipped to identify unknown attacks effectively. These models frequently misclassify malicious traffic from unknown categories as benign, thereby introducing serious security vulnerabilities [5], [6], [7], [8], [9], [10]. Consequently, there is an urgent need for models that not only excel in accurately classifying known traffic patterns but also possess robust mechanisms for detecting and identifying unknown malicious traffic. Addressing this challenge requires the development of advanced models capable of detecting unknown traffic, which is increasingly recognized as an essential component of modern network security frameworks.

Currently, methods for detecting unknown traffic can be broadly categorized into two approaches: **discriminative** [11], [12], [13], [14], [15], [16], [17] and **generative** methods [18], [19], [20], [21], [22], [23]. Discriminative methods primarily focus on learning decision boundaries between different categories by modeling the conditional probability $\mathbf{P}(\mathbf{Y} \mid \mathbf{X})$, which represents the probability of assigning a label $\mathbf{Y}$ to a sample $\mathbf{X}$. While effective at classification, these methods prioritize distinguishing between known categories without capturing the underlying data distribution. As a result, their reliance on decision boundaries may limit their ability to detect complex or unknown traffic patterns. Conversely, generative methods aim to model not only decision boundaries but also the data generation process, providing a holistic understanding of the distributional characteristics of known categories. By capturing the mechanisms behind data generation, these methods construct flexible models that accurately represent known categories while enabling the detection of unknown traffic. However, despite their advantages, generative methods are still constrained by two main limitations:

(i) **The distribution of known class representations is often not compact, while the space allocated for unknown**

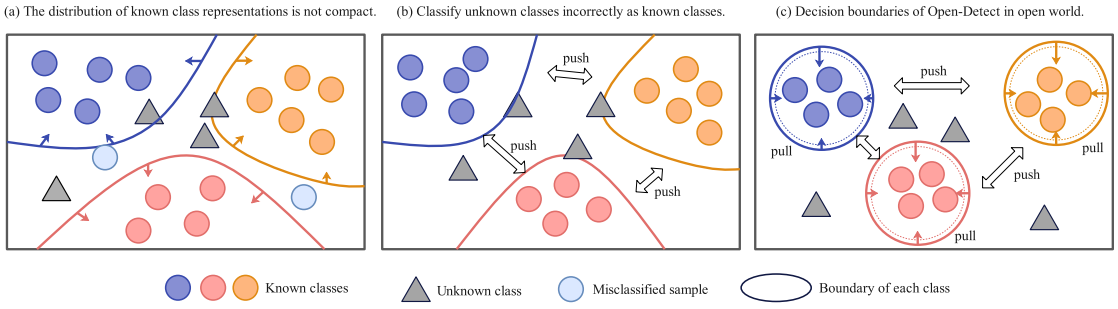


Fig. 1. Comparison of decision boundaries. In Scenario (a), the absence of intra-class compactness causes samples from known classes to overlap with other known classes, leading to frequent misclassifications. Scenario (b) demonstrates incremental improvement, with intra-class compactness partially achieved; however, insufficient inter-class separation still leads to significant misclassification errors, particularly when unknown classes are identified as known. Finally, Scenario (c) highlights the effectiveness of Open-Detect, which achieves both intra-class compactness and inter-class separation, reducing the likelihood of misclassifications for both known and unknown traffic.

classes may be too limited. As shown in Fig. 1 (a), the representation space of known classes traffic tends to be overly expansive, failing to achieve intra-class compactness. This results in insufficient similarity between samples of the same class. For example, while [19] attempts to address this issue by constructing a skewed dataset that restricts the input space for known classes and reserves space for unknown classes, this objective is not directly optimized during training. Consequently, the distribution of known classes remains inadequately compressed, which often leads to the misclassification of known class traffic.

(ii) **Models often misclassify unknown classes as known classes, leading to ineffective inter-class separation and reduced detection accuracy,** as illustrated in Fig. 1 (b). For instance, Zen-tor [20] and [21] address this by generating synthetic unknown traffic with GANs. However, the core challenge is that the distribution of unknown traffic is unknowable, as actual unknown traffic cannot be directly obtained, leading to high misclassification rates. Similarly, the author in [24] assumes normal data is tightly clustered while malicious data is dispersed, using this to synthesize unknown traffic. However, malicious data often overlaps with normal traffic, especially with TLS-encrypted traffic mimicking normal behavior, making differentiation and data augmentation more difficult and limiting detection accuracy improvement.

To address the two key challenges illustrated in Fig. 1 (c), an effective approach involves optimizing model performance by minimizing intra-class differences while maximizing inter-class differences. Minimizing intra-class differences helps cluster samples within the same class more tightly in the representation space, enhancing class distinction and reducing misclassification. This improves the model's accuracy in recognizing known categories and clarifies the boundaries between them. At the same time, maximizing inter-class differences boosts separation between known and unknown categories, enhancing the model's sensitivity to previously unseen or anomalous traffic. As a result, the distribution of known traffic categories becomes more compact, while the gap between known and unknown categories widens [25], [26]. This strategy helps the model better capture subtle differences between normal and abnormal traffic, ultimately improving detection accuracy, particularly in complex, real-world scenarios.

In this paper, we introduce Open-Detect, a model specifically designed for detecting unknown malicious attacks. Open-Detect leverages variational autoencoders (VAEs) and Gaussian prototypes to achieve fine-grained classification of known traffic categories and effective detection of unknown malicious traffic [27], [28]. We propose two key constraint mechanisms to achieve this goal: generative constraints and discriminative constraints. The generative constraints map traffic of known categories to Gaussian prototypes in the latent space, ensuring that samples from the same category are tightly clustered, thereby enhancing intra-class compactness and improving classification accuracy for known traffic. Conversely, the discriminative constraints enforce that the latent variables of each sample are as close as possible to the Gaussian prototype of its true label while simultaneously promoting separation between prototypes of different classes in the latent space. This promotes better inter-class separation, enhancing the model's ability to detect unknown traffic. By combining both generative and discriminative constraints, Open-Detect not only improves the identification of known traffic categories but also significantly enhances the model's capability to discriminate between unknown traffic classes, thereby boosting its generalization capability and robustness in real-world applications.

The main contributions of our work are summarized as follows:

- We propose a novel theoretical framework for detecting unknown malicious traffic, called Open-Detect. This framework integrates a variational autoencoder architecture with both generative and discriminative constraints, enabling effective detection of previously unseen malicious traffic.
- We theoretically prove that intra-class compactness in Open-Detect can reduce misclassification between known classes and lower the probability of misclassifying unknown class malicious traffic as known classes.
- Open-Detect improves the identification of unknown categories by learning multiple Gaussian prototypes to represent known classes within the latent space. This design ensures precise classification of known categories while maintaining robust detection capabilities for previously unseen malicious traffic.
- We rigorously evaluate the performance of Open-Detect on two public datasets, demonstrating that our proposed method outperforms existing state-of-the-art approaches in both closed- and open-world scenarios.

TABLE I
RELATED WORK: DISCRIMINATIVE AND GENERATIVE METHODS FOR UNKNOWN TRAFFIC DETECTION

	Year [ref]	Input	Technique	Applicability	Notes
Discriminative	2020 [15]	TI	CNN	DL	Threshold-based filtering of unknown traffic.
	2021 [14]	FF	Cluster labeling k-means	ML	A multitiered hybrid intrusion detection system.
	2021 [29]	PP	Siamese network	DL	Focusing on intra-class compactness and inter-class separability.
	2023 [30]	MAV	k-Logit neighbor distance-based	DL	A well-regularized model.
	2023 [31]	TAM	CNN	DL	Fully capture key informative features leaked from traces.
	2024 [32]	FV	Cost-sensitive matrix	DL	Filtering unknown classes based on output layer probability distributions.
	2024 [33]	FV	DNN	DL	Addressing the problem of co-existing unknown and adversarial attacks.
	2024 [13]	TI	Multi-binary classifier	DL	Leveraging large amounts of unlabeled data for model training.
Generative	2021 [22]	FF	VAE	DL	Minimizing empirical and identification risks.
	2022 [20]	FF	GAN	DL	Synthesizing unknown class traffic.
	2023 [19]	FF+PP	One-class classifiers	DL	Limiting the input space scope to known classes.
	2023 [21]	FF	GAN	DL	Synthesizing unknown class traffic.
	2023 [23]	FF	FSFDP method [34]	ML	A density-based heuristic clustering method.
	2024 [18]	FV	AE	DL	Also addresses incremental update issues.
	This work	TI	CVAE + Prototype Learning	DL	Based on Bayesian inference, with a complete theoretical framework.

Input: Flow Features (FF), Packet Payload (PP), Feature Vector (FV), Traffic Image (TI), Mean Activation Vector (MAV).
Technique: Convolutional Neural Networks (CNN), Generative Adversarial Networks (GAN), Autoencoders (AE), Conditional Variational Autoencoders (CVAE).

The remainder of this paper is organized as follows. In Sec. II, we review related work on unknown traffic detection. Sec. III presents a mathematical formulation of the problem and introduces an optimization method. In Sec. IV and V, we provide a detailed description of the proposed model and theoretical justification. Sec. VI discusses the experiments conducted and their results. Finally, Sec. VII and VIII offers a summary of the paper's discussion and conclusion.

II. RELATED WORK

Existing techniques for detecting unknown traffic can be broadly categorized into two primary types: generative unknown traffic detection methods and discriminative unknown traffic detection methods. In this section, we present a concise overview of key related works, which are summarized in Table I.

A. Discriminative Unknown Traffic Detection Methods

Discriminative models are designed to learn the categorization boundaries of data directly, enabling them to distinguish effectively between different classes. These models often employ classification layers specifically tailored for unknown traffic detection. One of the simplest techniques involves rejecting samples with low confidence based on the output probability from the SoftMax layer [15]. However, such threshold-based approaches rely heavily on expert-defined thresholds, making them time-consuming, labor-intensive, and prone to biases. This reliance on thresholds also introduces instability in detecting unknown traffic, particularly in dynamic or unpredictable network environments.

The work presented in [32] addresses the challenge of unknown traffic detection in the presence of class imbalance by leveraging cost-sensitive matrices and the prediction confidence of Deep Neural Networks (DNNs) on samples. In contrast, [15] introduces an automated learning framework that updates the training dataset by distinguishing between known and unknown traffic based on confidence levels. Signature-based intrusion detection systems (IDS) and anomaly-based IDS are specifically engineered to secure vehicular networks. The anomaly-based IDS employs two biased classifiers to minimize false negatives (FNs) and false positives (FPs), thereby enhancing the detection rate (DR) and reducing the false alarm rate (FAR) [14]. The SEEN framework [35] organizes unknown traffic detection into three stages: discovery, clustering, and model updating. GradBP [36] leverages gradient backpropagation to effectively detect zero-day applications. Furthermore, [30] demonstrates that robust regularization can significantly enhance model performance, proposing three methodologies grounded in k-logit neighbor distances. Reference [31] develops a robust traffic representation method that generates a Traffic Aggregation Matrix (TAM) using packet counts and time series to fully capture key information features leaked in the traffic. Reference [33] integrates multiple machine learning/deep learning models into a single framework using heterogeneous ensemble learning, addressing the coexistence of unknown attacks and adversarial attacks in fine-grained attack detection scenarios.

B. Generative Unknown Traffic Detection Methods

Generative models are designed to learn the distribution of known data classes, enabling them to generate and categorize data based on this learned representation. Many generative methods leverage probabilistic graphical models and autoencoders. However, a common limitation of these models is their inability to effectively differentiate between known and unknown classes [37].

The Trident framework [18] employs multiple autoencoder-based single-class classifiers to achieve granular detection of unknown traffic and facilitate incremental updates to the response model. CVAE-EVT [22] frames the fine-grained known/unknown intrusion detection problem as a two-stage minimization problem. The first stage involves finding a scoring measure that minimizes the empirical risk of

misclassifying known attacks. The second stage uses Extreme Value Theory (EVT) to model the distribution of reconstruction errors to distinguish between known and unknown attacks. Similarly, the study in [23] introduces a density-based clustering algorithm inspired by the FSFDP method [34] to detect unknown attacks. To address the challenges posed by a single threshold, [38] develops precise decision boundaries tailored to each known class. In scenarios lacking prior information about unknown attacks, [20] and [24] leverage neural networks to synthesize unknown traffic, effectively converting the open-world problem into a closed-world classification challenge.

C. Summary

The aforementioned studies have significantly advanced the field of unknown traffic detection, offering valuable insights and methodologies. However, they often fail to address the issue of known classes not being compactly distributed within the representation space. This lack of compactness can lead to insufficient allocation of space for unknown classes, increasing the risk of misclassifying unknown classes as known ones.

III. Problem Statement and Modeling

A. Problem Statement

We define the unknown traffic detection problem as follows: given a labeled training set $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$ with N instances and C known classes ($y_i \in Y = \{1, 2, \ldots, C\}$), the objective is to learn a model $f : D \rightarrow \{1, 2, \ldots, C, C+1\}$ that can classify instances in a test set with potential unknown classes. The model f has two objectives: (1) to classify known classes accurately, and (2) to assign unknown instances to class $C + 1$, effectively distinguishing them as unknown traffic.

B. Problem Modeling

Theoretically, we formulate the problem of unknown traffic detection uniformly as

$$\underset{F}{\operatorname{argmax}} \left[I\left(F; C_{known}\right) - \alpha H\left(C_{unknown} | F\right) \right], \tag{1}$$

where F represents a specific feature of the network flow, C denotes the traffic classes, $I(;)$ indicates the mutual information, and $H(\cdot)$ denotes entropy. By learning an effective feature representation that maximizes the mutual information within known classes while minimizing the uncertainty of unknown classes, the model can accurately identify and reject samples belonging to the open world.

As shown in Eq. 1, this formula aims to optimize the feature representation such that: (1) It maximizes the mutual information with the known classes, ensuring that the features are well aligned with the traffic patterns of the known classes. (2) It minimizes the entropy of the unknown classes, ensuring that the model can confidently reject flows that do not belong to any of the known traffic classes, thus improving the model's ability to handle open-world samples. In this paper, we transform the optimization problem of Eq. 1 into a problem of minimizing **Observed Risk** and **Unknown Risk**.

In the context of unknown traffic detection, for known classes of traffic D, $(x, y) \sim P$, $f(x; \theta) \in \mathbb{R}^Y$, our goal is to minimize Observed Risk $R_{observed}$,

$$R_{observed} = \min_{\theta} \underset{(x,y)\sim P}{\mathrm{E}} L\left(f\left(x; \theta\right), y\right), \tag{2}$$

approximated by the training data D as:

$$R_{observed} = \min_{\theta} \frac{1}{N} \sum_{i=1}^{N} L\left(f\left(x_i; \theta\right), y_i\right). \tag{3}$$

For unknown class traffic, we define the unknown space and the unknown risk as follows, using the known class training samples D, the unknown space can be defined as $\Omega = S - \cup_{x \in D} \sigma(x)$, where σ represents the local feature space generated by the training samples x, and S encompasses all the unknown spaces and the remaining space. Consider a measurable recognition function f, where $f(x) = 1$ for recognition of the class y of interest and $f(x) = 0$ when y is not recognized. The unknown risk can be defined as

$$R_{unknown}\left(f\right) = \frac{\int_{\Omega} f\left(x; \theta\right) dx}{\int_{S} f\left(x; \theta\right) dx}. \tag{4}$$

Indeed, the purpose of minimizing the observation risk $R_{observed}$ is to ensure the accurate classification of known classes while minimizing the unknown risk $R_{unknown}$ aims to reduce the likelihood of the model misclassifying unknown classes as known, thereby improving the overall performance of the model in open-world scenarios. However, $R_{observed}$ and $R_{unknown}$ are closely related; minimizing the observation risk contributes to reducing the unknown risk. The joint optimization objective can thus be defined as

$$\underset{f}{\operatorname{argmin}} \{R_{unknown}\left(f\right) + \eta R_{observed}\left(f\left(D\right)\right)\}, \tag{5}$$

where η is a regularization constant. It can be observed that Eq. 5 has the same meaning as Eq. 1.

The key to solving the problem of classifying known classes and detecting unknown classes lies in identifying a measurable recognition function f, with f defined as in Eq. 5. Since unknown and observed risks are coupled to each other in the function f, an effective f must characterize not only the boundaries that classify known classes, but also the boundaries that distinguish between known and unknown classes.

This formulation introduces a dual challenge: first, finding a suitable recognition function f that can accurately identify unknown categories while maintaining the classification accuracy of known categories, and second, addressing the difficulty of selecting an appropriate regularization constant to balance the trade-off between the unknown risk and the observed risk. This trade-off is particularly challenging in practice due to the coupled nature of the two risks. To provide sufficient flexibility and decouple the two risks, two separate functions are employed [22].

$$\begin{aligned} g_1^* &= \underset{g_1 : x \rightarrow y \in Y}{\operatorname{argmin}} R_{observed}\left(g_1\left(x\right)\right) \\ &= \underset{g_1 : x \rightarrow y \in Y}{\operatorname{argmin}} \int_{X \times Y} L\left(x, y, g_1\left(x\right)\right) p\left(x, y\right) dxdy \\ &= \underset{g_1 : x \rightarrow y \in Y}{\operatorname{argmin}} \int_{X \times Y} L\left(x, y, g_1\left(x\right)\right) \left[\lambda p\left(x|y\right) p\left(y\right) \right. \\ &\quad \left. + \left(1 - \lambda\right) p\left(y|x\right) p\left(x\right) \right] dxdy, \end{aligned} \tag{6}$$

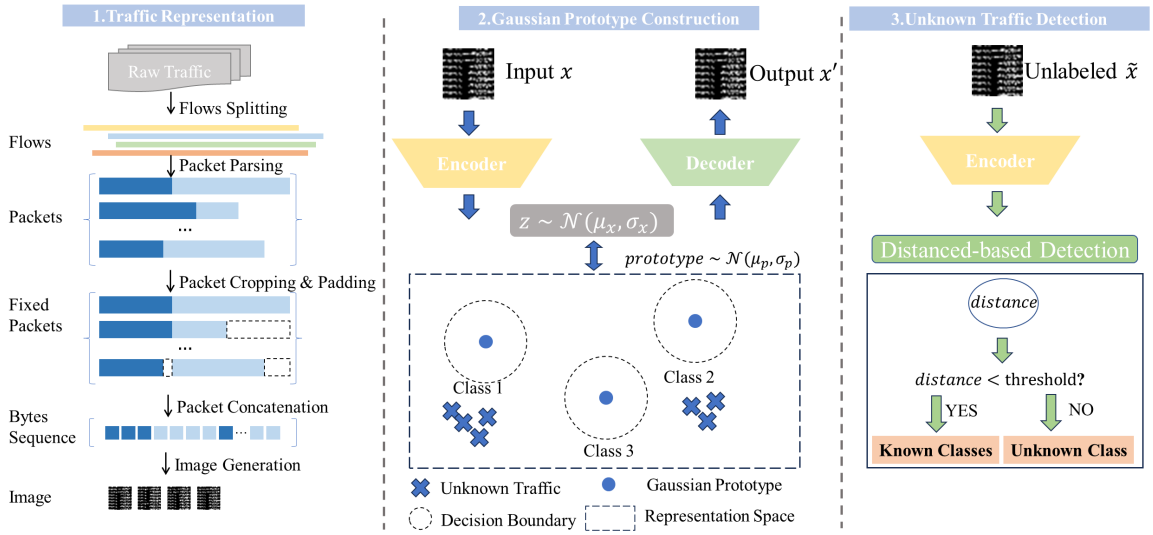


Fig. 2. Overview of the Open-Detect framework for unknown network traffic detection.

where $L(x, y, g(x))$ is the loss function defining the penalty for misclassified samples x,

$$L(x, y, g(x)) = \begin{cases} 0, & g(x) = y \\ 1, & g(x) \neq y \end{cases}, \tag{7}$$

$p(x, y)$ denotes the joint distribution of samples and labels, and the following decomposition is done in Eq. 6:

$$p(x, y) = \lambda p(x|y)\, p(y) + (1 - \lambda)\, p(y|x)\, p(x). \tag{8}$$

Based on Eq. 6,

$$g_2^*(x) = \underset{g_2: x \to y \in Y \cup \{C+1\}}{\operatorname{argmin}} R_{unknown}\left(g_2 | y = g_1^*(x)\right). \tag{9}$$

Based on Eq. 6 and 9, the optimization objective defined in Eq. 5 translates into modeling $p(x, y)$, which effectively involves modeling $p(x|y)$ and $p(y|x)$. Given the inherent encoder-decoder structure of autoencoders, we chose to implement the model using VAEs.

IV. Our Approach

This section details the proposed Open-Detect model.

A. Overview

To address the challenges of detecting unknown network traffic, we propose a method using Gaussian prototypes. As illustrated in Fig. 2, Open-Detect consists of three principal components: a feature extraction module, a Gaussian prototype construction module, and an unknown traffic detection module. The feature extraction module processes network flow data by converting the raw bytes of the flow into a grayscale image representation. This transformation preserves the rich features inherent in the network flow while enabling further analysis. The Gaussian prototype construction module utilizes a CVAE, which maps each known traffic class to a Gaussian prototype in the latent space. This method achieves both inter-class separation and intra-class compactness. For intra-class compactness, a generative model is trained with generative constraints, minimizing $p(x|y)$ to obtain latent variables z with representative and discriminative features. For inter-class separation, discriminative constraints are applied to ensure that Gaussian prototypes of different traffic classes are well-separated in the latent space. This is accomplished by learning the intrinsic distributions of the known class features z and minimizing $p(y|x)$. The unknown traffic detection module employs distance-based metrics. If the distance between a given network flow and the nearest Gaussian prototype in the latent space exceeds a predefined threshold, the flow is classified as unknown. Conversely, if the flow is closer to a known Gaussian prototype, it is classified as the corresponding known traffic class.

B. Feature Extraction

To capture the fine-grained behavior of encrypted network traffic, we convert raw traffic into grayscale images [39], [40]. First, we organize the traffic into flows, each comprising packets of a specific protocol. Packets within a flow carry essential information about interactions between two hosts, including connection establishment, data transfer, and overall communication patterns.

Flow-level features are critical for characterizing network behavior and enhancing traffic classification. To preserve relevant information while reducing noise, we exclude non-IP protocols such as ARP and DHCP and strip the Ethernet frame header from the data link layer. Each packet is divided into a header and a payload: the header covers the network and transport layers, while the payload, immediately following the transport header, consists entirely of application-layer data. Notably, payload data is not always encrypted; for instance, Client Hello and Server Hello packets contain key plaintext information from the TLS handshake.

For a flow $l = (l_1, l_2, \ldots, l_n)$, we select the first k packets and extract a specific number of bytes from each for further processing. Specifically, we take the first k_1 bytes from the packet's IP header and the first k_2 bytes from the payload. If the number of bytes is insufficient, we pad the gap with the value 0×00. Additionally, to mask the IP address field, we replace it with the value 0.0.0.0. The total number of bytes used for each flow is calculated as $L = k * (k_1 + k_2)$. These bytes are then converted into integers ranging from 0 to 255 for

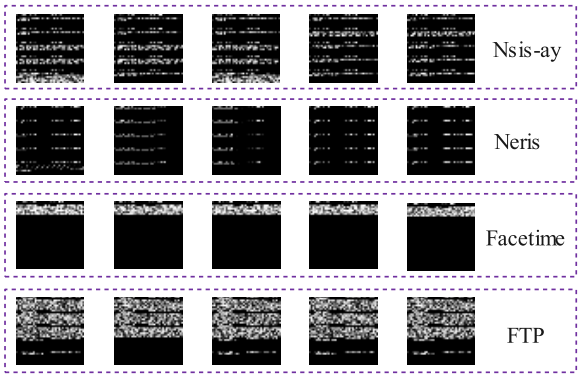


Fig. 3. Grayscale-based representation of network flows.

subsequent analysis. In this paper, we define L as the number of squares, which is used to convert a sequence of bytes of length L into a two-dimensional grayscale image [41], [42], [43]. The parameters k, k_1, and k_2 are hyperparameters, and we set their values to 8, 80, and 48, respectively.

Related work shows that machine learning and deep learning methods can effectively extract latent information from ciphertext: early studies classified ciphertext sequences by directly feeding them into RNNs [44]; later works improved classification performance by constructing ciphertext features or converting ciphertext into images, utilizing models such as SVM, CNN, and CNN-Transformer [45]. ET-BERT [46] transforms encrypted traffic data into a format suitable for BERT pretraining through tokenization and other preprocessing steps, learning statistical patterns and semantic representations. Overall, these studies support the feasibility of converting network flows (including both plaintext and ciphertext) into grayscale images for analysis. Notably, while the converted grayscale images contain some ciphertext information, deep learning models can still learn the differences between various cryptographic components and parameters [47]. Therefore, retaining part of the ciphertext in the payload is justified, as confirmed in the subsection VI-E.

As shown in Fig. 3, which illustrates the effect of converting different traffic classes into images, there are noticeable differences between the images corresponding to various traffic classes. In the figure, white dots typically represent pixel values of 255, indicating the brightest areas of the image, while black dots represent pixel values of 0, indicating the darkest areas. Intermediate gray values correspond to varying brightness levels between 0 and 255. The effective representation of network traffic is a prerequisite for ensuring the effectiveness of the model.

C. Gaussian Prototype Construction

We use the CVAE to learn latent distribution features within a supervised learning framework. The CVAE combines variational Bayesian inference with an encoder-decoder architecture to effectively capture these features. As illustrated in the Fig. 2, the Gaussian prototype construction module consists of an encoder, decoder, and multiple Gaussian prototypes. The encoder maps known class distributions into the latent space to extract features, while the decoder generates samples from this space to approximate the original distribution. The Gaussian prototypes represent each known class in the latent space.

Specifically, we consider the use of negative log-likelihood to model the distribution of a known class of samples $p_\theta(x, y)$,

$$Loss = \underset{x,y\sim D}{E}\left(-\log p_\theta(x, y)\right), \tag{10}$$

where

$$\log p_\theta(x, y) = \lambda \log p_\theta(x|y)\, p(y) + (1-\lambda)\log p_\theta(y|x)\, p(x). \tag{11}$$

The hyperparameter λ is used to balance these two important terms. Specifically, $p_\theta(x|y)$ is associated with the generation task and represents the generative constraints of the model given the label y, while $p_\theta(y|x)$ is linked to the classification task and represents the discriminative constraints of the model given the sample x [48].

1) Generative Constraints: The objective of the model's generative constraints is to approximate $p_\theta(x|y)$, given the label y, in a generative manner to produce a more efficient feature representation. A standard CVAE maps each known class to a Gaussian prototype in the latent space, ensuring that samples from the same class are tightly clustered, thereby promoting intra-class compactness. Similar to traditional CVAE-based methods, we optimize $p_\theta(x|y)$ using optimal variational inference to derive the following Evidence Lower Bound (ELBO) function [49]:

$$\begin{aligned} L_{ELBO} &= \log p_\theta(x|y) = \log \int p_\theta(x, z|y)\, dz \\ &= \log \int q_\phi(z|x, y)\, \frac{p_\theta(x, z|y)}{q_\phi(z|x, y)} dz \\ &\geq E_{q_\phi(z|x,y)}\left(\log\left(\frac{p_\theta(x, z|y)}{q_\phi(z|x, y)}\right)\right), \end{aligned} \tag{12}$$

where $q_\phi(z|x, y)$ represents the inference process and $p_\theta(x, z|y)$ represents the generation process.

$q_\phi(z|x, y) = q_\phi(z|x)$, where $x \in Prototype_y$. In the inference process, $q_\phi(z|x)$ represents the encoded feature of the sample x, which forms a Gaussian distribution in the latent space. This encoding is achieved by the encoder using the reparameterization trick [49].

We have $p_\theta(x, z|y) = p(z|y)\, p_\theta(x|z)$. In the generation process, $p(z|y) \sim \mathcal{N}\left(z; \boldsymbol{\mu_y}, \boldsymbol{I}\right)$, represents sampling from a Gaussian distribution, where $\boldsymbol{\mu_y}$ is the mean associated with the class y. The term $p_\theta(x|z)$ denotes the decoder, which reconstructs the input samples from the latent variable z.

Therefore, we have

$$\begin{aligned} L_{ELBO} &\geq E_{q_\phi(z|x,y)}\left(\log\left(\frac{p_\theta(x, z|y)}{q_\phi(z|x, y)}\right)\right) \\ &= E_{q_\phi(z|x)} \log p_\theta(x|z) - E_{q_\phi(z|x)} \log \frac{q_\phi(z|x)}{p(z|y)} \\ &= E_{q_\phi(z|x)} \log p_\theta(x|z) - KL\left(q_\phi(z|x) \,\|\, p(z|y)\right) \\ &= L_1 - L_2, \end{aligned} \tag{13}$$

where $x \in Prototype_y$.

The first term, L_1, represents the reconstruction loss:

$$L_1 = E_{q_\phi(z|x)} \log p_\theta(x|z) = \left\| x' - x \right\|_2, \tag{14}$$

which measures how well the decoder reconstructs the input samples, encouraging the encoded feature z to capture rich representations of x.

The second term, L_2, represents the KL divergence between the approximate posterior and the prior over the latent variable z:

$$\begin{aligned} L_2 &= KL\left(q_\phi(z|x) \,\|\, p(z|y)\right) \\ &= KL\left(\mathcal{N}(z;\mu_x,\sigma_x) \,\|\, \mathcal{N}\left(z;\boldsymbol{\mu_y},\boldsymbol{I}\right)\right) \\ &= \frac{1}{2}\sum_{i=1}^{d}\left[\left(\mu_i(x)-\mu_i^y\right)^2+\sigma_i^2-\log\sigma_i^2-1\right], \end{aligned} \tag{15}$$

where d is the dimension of the latent space, which ensures that the latent variable z follows a Gaussian distribution. Moreover, it encourages z to be close to the Gaussian prototype associated with the label y, thus promoting intra-class compactness.

2) Discriminative Constraints: Intuitively, the generative constraints ensure that the latent variable z remains close to the Gaussian prototype corresponding to its label. This promotes a compact intra-class distribution, where samples of the same class are tightly clustered in the latent space. Simultaneously, inter-class separation is crucial to ensure that the Gaussian prototypes of different classes are sufficiently distinct. This separation enhances the model's ability to accurately detect unknown or outlier traffic by capturing meaningful class boundaries within the latent space.

In contrast, the discriminative constraints aim to keep the latent variable z away from the Gaussian prototypes of different classes. Essentially, while the generative constraints focus on promoting intra-class compactness, the discriminative constraints enforce inter-class separation. Like the generative constraints, these discriminative constraints can be implemented through Bayesian inference, further guiding the latent variable to effectively distinguish between classes.

We have:

$$\begin{aligned} \log q_\phi(y|x) &= \log\int q_\phi(y,z|x)\,dz \\ &= \log\int q_\phi(z|x)\,q(y|z)\,dz, \end{aligned} \tag{16}$$

where $q_\phi(z|x)$ represents the encoded feature of the sample x, forming a Gaussian distribution in the latent space through the encoder using the reparameterization trick. $q(y|z)$ is the distribution probability of the latent variable z across the C Gaussian prototypes. The closer z is to the Gaussian prototype, the higher the probability of y being assigned to that class.

Since both the latent variable z and the Gaussian prototype follow a Gaussian distribution, we define the following probability formula:

$$\begin{aligned} q(y|z) &= q(z \in Prototype_i) \\ &= \frac{\exp\left(\gamma\left(-dis\left(z,Prototype_i\right)\right)\right)}{\sum_{k=1}^{C}\exp\left(\gamma\left(-dis\left(z,Prototype_k\right)\right)\right)}, \end{aligned} \tag{17}$$

where $dis(z,Prototype_i) = KL\left(\mathcal{N}(\mu_x,\sigma_x)\|\mathcal{N}\left(\boldsymbol{\mu_y},\boldsymbol{I}\right)\right)$, γ is a hyperparameter. If and only if the two distributions are identical, the KL divergence is equal to zero and is always non-negative.

Considering the above, we have:

$$\log q_\phi(y|x) = \log\frac{\exp\left(\gamma\left(-dis\left(z,Prototype_i\right)\right)\right)}{\sum_{k=1}^{C}\exp\left(\gamma\left(-dis\left(z,Prototype_k\right)\right)\right)}, \tag{18}$$

Algorithm 1 The Training Process of Open-Detect

Input: Set of labeled data D, hyperparameters λ, γ, d, k, k_1, k_2,

Output: The encoder $q_\phi(z|x)$ and the decoder $p_\theta(x|z)$.

1: Convert raw network flows into grayscale images.
2: Calculate the generative constraints loss L_{ELBO} according to Eq. 13.
3: Compute the discriminative constraints loss based on Eq. 18.
4: Calculate the overall loss $Loss$ using Eq. 20, and update the network parameters.

where $z \in q_\phi(z|x)$. In summary, minimizing $q_\phi(y|x)$ encourages the latent variable z to move closer to the Gaussian prototype of its corresponding class, while simultaneously increasing the distance to the prototypes of other classes. This optimization process effectively promotes inter-class separation.

Since $p_\theta(y|x)$ is intractable to compute, we use variational inference with $q_\phi(y|x)$ to approximate $p_\theta(y|x)$. The terms $p(y)$ and $p(x)$ are independent of the model parameters and can be omitted. Therefore, we have:

$$\begin{aligned} \log p_\theta(x,y) &= \lambda\log p_\theta(x|y)\,p(y) + (1-\lambda)\log p_\theta(y|x)\,p(x) \\ &\simeq \lambda\log p_\theta(x|y) + (1-\lambda)\log q_\phi(y|x), \end{aligned} \tag{19}$$

Consequently, the overall loss function is:

$$\begin{aligned} Loss &= \underset{x,y\sim D}{E}\left(-\log p_\theta(x,y)\right) \\ &\simeq \lambda\underset{x,y\sim D}{E}\left(-\log p_\theta(x|y)\right) \\ &\quad + (1-\lambda)\underset{x,y\sim D}{E}\left(-\log q_\phi(y|x)\right), \end{aligned} \tag{20}$$

where the hyperparameter λ controls the trade-off between these two objectives. The training process of Open-Detect is shown in Algorithm 1.

D. Unknown Traffic Detection

Once the generative and discriminative constraints are satisfied, the encoder, which approximates $q_\phi(z|x)$, and the decoder, which approximates $p_\theta(x|z)$, are trained. Simultaneously, the parameters of the Gaussian prototypes corresponding to each class are learned. The detection of unknown traffic is based on distance, achieved by calculating the distance between the latent variable z and the Gaussian prototypes. The following steps are performed to detect unknown traffic: (i) The grayscale image x of the flow is obtained through the feature extraction module. (ii) The corresponding latent space representation z is obtained via $q_\phi(z|x)$. (iii) The distance $dist$ from the latent variable z to the nearest Gaussian prototype is computed:

$$dist = \min_{Prototype_i} dis\left(z,Prototype_i\right). \tag{21}$$

The predicted label y_{pred} is determined as follows,

$$y_{pred} = \begin{cases} \text{class } i, & dist < threshold \\ \text{unknown}, & dist \geq threshold \end{cases}, \tag{22}$$

Here, the *threshold* is set to ensure that 95% of the samples in the validation set are classified into known classes.

V. Theoretical Justification

In this section, we provide a theoretical proof that intra-class compactness can reduce the misclassification error of known classes, as well as the misclassification error of unknown class.

Theorem 1: Let $d_{ij} = \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|$ denote the distance between prototypes of classes y_i and y_j, and σ_x^2 be the latent variable variance. The probability of misclassification satisfies:

$$P_{\text{err}}^{i\to j} \le \exp\left(-\frac{d_{ij}^2}{8\sigma_x^2}\right). \tag{23}$$

Proof: Consider a sample x from class y_i, whose latent variable follows:

$$z = \boldsymbol{\mu}_i + \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma_x^2 \boldsymbol{I}). \tag{24}$$

Misclassification to class y_j occurs when:

$$\|\boldsymbol{\mu}_i + \epsilon - \boldsymbol{\mu}_j\|^2 < \|\epsilon\|^2. \tag{25}$$

Expanding the left-hand side and simplifying:

$$\begin{aligned}(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j + \epsilon)^\top(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j + \epsilon) &< \epsilon^\top\epsilon \\ \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2 + 2\epsilon^\top(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j) &< 0.\end{aligned} \tag{26}$$

This yields the necessary condition for misclassification:

$$\epsilon^\top(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j) < -\frac{1}{2}\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2. \tag{27}$$

Define $\Delta\boldsymbol{\mu} = \boldsymbol{\mu}_i - \boldsymbol{\mu}_j$. The left-hand side constitutes a linear combination of Gaussian variables:

$$\epsilon^\top\Delta\boldsymbol{\mu} \sim \mathcal{N}\left(0, \sigma_x^2\|\Delta\boldsymbol{\mu}\|^2\right). \tag{28}$$

Standardizing this random variable:

$$\eta = \frac{\epsilon^\top\Delta\boldsymbol{\mu}}{\sigma_x\|\Delta\boldsymbol{\mu}\|} \sim \mathcal{N}(0,1). \tag{29}$$

The misclassification condition transforms to:

$$\eta < -\frac{\|\Delta\boldsymbol{\mu}\|}{2\sigma_x}. \tag{30}$$

Applying the tail bound for the standard normal distribution [50]:

$$\Phi(-t) \le \frac{1}{\sqrt{2\pi}t}e^{-t^2/2} \le e^{-t^2/2}, \quad t > 0. \tag{31}$$

The second inequality holds when $t > \frac{1}{\sqrt{2\pi}} \approx 0.3989$. In OpenDetect, where the latent space compactness $\sigma_x \to 0$ and inter-class separation d_{ij} increases, the parameter $t = \frac{\|\Delta\boldsymbol{\mu}\|}{2\sigma_x}$ naturally satisfies $t \gg 0.3989$ (empirically verifiable), thus validating the inequality scaling.

Substituting $t = \frac{\|\Delta\boldsymbol{\mu}\|}{2\sigma_x}$ gives:

$$P_{\text{err}}^{i\to j} = \Phi\left(-\frac{\|\Delta\boldsymbol{\mu}\|}{2\sigma_x}\right) \le \exp\left(-\frac{\|\Delta\boldsymbol{\mu}\|^2}{8\sigma_x^2}\right). \tag{32}$$

■

From Theorem 1, it can be seen that as the distance between prototypes, $\Delta\boldsymbol{\mu}$, increases and the intra-class variance, σ_x^2, decreases, the misclassification error of known classes also decreases.

Theorem 2: Let $z_u \sim \mathcal{N}(\boldsymbol{\mu}_u, \sigma_u^2\boldsymbol{I})$ denote the latent variable of an unknown class sample x_u, and τ_k be the detection threshold for known class y_k satisfying:

$$P_{z\sim\mathcal{N}(\boldsymbol{\mu}_k,\sigma_k^2\boldsymbol{I})}\left(\|z - \boldsymbol{\mu}_k\|^2 \le \tau_k\right) \ge 1 - \alpha, \tag{33}$$

where α represents the significance level (typically $\alpha = 0.05$). The upper bound on misclassification probability to known classes satisfies:

$$P_{\text{unk}\to\text{known}} \le \sum_{k=1}^{C}\exp\left(-\frac{(\|\boldsymbol{\mu}_u - \boldsymbol{\mu}_k\|^2 - \tau_k)^2}{8\sigma_u^2\|\boldsymbol{\mu}_u - \boldsymbol{\mu}_k\|^2}\right). \tag{34}$$

Proof: Misclassification occurs when an unknown sample satisfies the decision criterion for any known class y_k:

$$\|z_u - \boldsymbol{\mu}_k\|^2 \le \tau_k. \tag{35}$$

Substituting the latent variable decomposition $z_u = \boldsymbol{\mu}_u + \epsilon_u$ where $\epsilon_u \sim \mathcal{N}(0, \sigma_u^2\boldsymbol{I})$:

$$\|\boldsymbol{\mu}_u - \boldsymbol{\mu}_k + \epsilon_u\|^2 \le \tau_k. \tag{36}$$

Expanding the quadratic form yields:

$$\|\boldsymbol{\mu}_u - \boldsymbol{\mu}_k\|^2 + 2\epsilon_u^\top(\boldsymbol{\mu}_u - \boldsymbol{\mu}_k) + \|\epsilon_u\|^2 \le \tau_k. \tag{37}$$

Under the practical assumptions that: (1) inter-class distance $d_{uk} = \|\boldsymbol{\mu}_u - \boldsymbol{\mu}_k\|$ significantly exceeds $\sqrt{\tau_k}$, and (2) the unknown class distribution is compact (σ_u^2 small), we can ignore the second-order term $\|\epsilon_u\|^2$ to obtain the dominant condition:

$$2\epsilon_u^\top(\boldsymbol{\mu}_u - \boldsymbol{\mu}_k) \le \tau_k - d_{uk}^2. \tag{38}$$

Rearranging terms reveals:

$$\epsilon_u^\top(\boldsymbol{\mu}_k - \boldsymbol{\mu}_u) \ge \frac{d_{uk}^2 - \tau_k}{2}. \tag{39}$$

Let $\Delta\boldsymbol{\mu} = \boldsymbol{\mu}_k - \boldsymbol{\mu}_u$. The left-hand side represents a zero-mean Gaussian variable:

$$\epsilon_u^\top\Delta\boldsymbol{\mu} \sim \mathcal{N}(0, \sigma_u^2\|\Delta\boldsymbol{\mu}\|^2). \tag{40}$$

Standardizing this quantity:

$$\eta = \frac{\epsilon_u^\top\Delta\boldsymbol{\mu}}{\sigma_u\|\Delta\boldsymbol{\mu}\|} \sim \mathcal{N}(0,1). \tag{41}$$

The misclassification condition becomes:

$$\eta \ge \frac{d_{uk}^2 - \tau_k}{2\sigma_u d_{uk}}. \tag{42}$$

Substituting $d_{uk} = \|\Delta\boldsymbol{\mu}\|$, we express the threshold as:

$$\eta \ge \frac{d_{uk}}{2\sigma_u} - \frac{\tau_k}{2\sigma_u d_{uk}}. \tag{43}$$

Applying the Gaussian tail bound with parameter $t = \frac{d_{uk}}{2\sigma_u} - \frac{\tau_k}{2\sigma_u d_{uk}}$ yields:

$$P_{\text{unk}\to\text{k}} \le \exp\left(-\frac{1}{2}\left(\frac{d_{uk}}{2\sigma_u} - \frac{\tau_k}{2\sigma_u d_{uk}}\right)^2\right). \tag{44}$$

Simplifying the exponent:

$$P_{\text{unk}\to\text{k}} \le \exp\left(-\frac{(d_{uk}^2 - \tau_k)^2}{8\sigma_u^2 d_{uk}^2}\right). \tag{45}$$

Finally, applying the union bound (Boole's inequality) across all C known classes [51]:

$$P_{\text{unk}\to\text{known}} \le \sum_{k=1}^{C}\exp\left(-\frac{(d_{uk}^2 - \tau_k)^2}{8\sigma_u^2 d_{uk}^2}\right). \tag{46}$$

■

TABLE II
DATASET SUMMARY OF USTC-TFC2016 AND MALICIOUS_TLS

ID	Class	# of flows	ID	Class	# of flows	ID	Class	# of flows	ID	Class	# of flows
U0	Gmail	2000	U12	Virut	2000	M0	nessus	2000	M12	PandaZeuSCC	2786
U1	FTP	2000	U13	Geodo	2000	M1	Panda.BZA!tr	1315	M13	arachni	2000
U2	Nsis-ay	2000	U14	MySQL	2000	M2	Shifu	3000	M14	Caphaw.A	3000
U3	Facetime	2000	U15	Htbot	2000	M3	TrickBotCC	3000	M15	golistmero	2000
U4	Weibo	2000	U16	Tinba	2000	M4	Drixed	3000	M16	awvs-v11	1148
U5	Cridex	2000	U17	Skype	2000	M5	Banker	2453	M17	DridexCC	3000
U6	Zeus	2000	U18	Miuref	2000	M6	Dynamer!ac	3000	M18	burpsuite	2000
U7	SMB	2000	U19	Neris	79	M7	Totbrick	2721	M19	Tiggre	3126
U8	BitTorrent	2000				M8	CobaltStrike	3004	M20	Vawtrak	3000
U9	WorldOfWarcraft	2000				M9	Qakbot	1352	M21	BuerLoader	3000
U10	Shifu	2000				M10	awvs-v12	701	M22	Caphaw.AH	3000
U11	Outlook	2000				M11	Tor	2978	M23	Upatre	1234

Note: Traffic IDs starting with "U" are from the USTC-TFC2016 dataset, while application IDs starting with "M" are from the Malicious_TLS dataset.

From Theorem 2, it can be seen that intra-class compactness reduces the misclassification probability through a dual mechanism. First, by compressing the variance of known classes ($\sigma_k \to 0$), the threshold $\tau_k \to 0$, which increases the effective distance of $d_{uk}^2 - \tau_k$. Secondly, by increasing the inter-class distance d_{uk}, the exponential decay rate accelerates, making $P_{\text{unk}\to\text{known}} \to 0$.

VI. EXPERIMENTAL ANALYSIS

In this section, we present and discuss our experimental results, focusing on the following questions:

- **RQ1.** How does the Open-Detect model perform in classifying known class traffic?
- **RQ2.** How effective is the Open-Detect model in detecting unknown malicious traffic?
- **RQ3.** Does Open-Detect perfectly achieve intra-class compactness and inter-class separation?

A. Experiment Settings

1) Datasets: In our experiments, we utilize two public datasets: USTC-TFC2016[1] and Malicious_TLS.[2] The raw pcap traffic from these datasets is processed using the method described in the feature extraction subsection, transforming the traffic into images for model construction and training. Details of the datasets are summarized in Table II.

- **USTC-TFC2016**: This dataset comprises two primary categories: normal and abnormal traffic. The normal traffic includes ten categories (e.g., Gmail, FTP), while the abnormal traffic also contains ten categories (e.g., Cridex, Geodo).
- **Malicious_TLS**: This dataset contains 24 different categories of cyberattacks collected from real networks between 2018 and 2021. All traffic in this dataset is encrypted using TLS.

2) Implementation Details: All experiments are performed on an Ubuntu 20.04 system equipped with an Intel Xeon Gold 5218R CPU@2.10GHz 80GB RAM and a Tesla V100 32GB GPU. Python 3.10.13 and Pytorch 2.1.1 are used to build our model. We used Resnet18 [52] as the encoder, paired with a mirror ResNet18 structure as the decoder. Data augmentation techniques included random center cropping and random level flipping. The same hyperparameters were applied across all datasets: $\lambda = 0.005$, $\gamma = 1$, $d = 128$, $k = 8$, $k_1 = 80$, $k_2 = 48$, and the Adam optimizer. We divided the dataset into training, validation, and testing in the ratio of 8:1:1.

3) Methods in Evaluation: We compared Open-Detect with eight baseline models, encompassing both closed-world and open-world methods. The details of these baseline models are as follows:

(i) Closed-world methods:

- **ET-BERT** [46] is based on the Transformer model, pretraining contextualized datagram-level representations from large-scale unlabeled data and fine-tuned with a small amount of labeled data.
- **IIT** [53] is designed with intra- and inter-flow attention modules to fully exploit intra- and inter-flow relationships for accurate identification of Decentralized applications.
- **FastTraffic** [54] uses the raw bytes in the packet as input and employs only three layers of MLP to achieve fast classification.
- **GraphDApp** [55] converts the packet length sequence of a DApp network flow into a traffic interaction graph, transforming the classification problem into a graph classification task, and utilizing a graph neural network-based classifier with an MLP.
- **ACID** [56] employs low-dimensional embeddings generated by a lightweight neural network equipped with multiple kernel layers. This approach effectively distinguishes between different classes in malware traffic classification using adaptive clustering techniques.

(ii) Open-world methods:

- **RoFi** [31] enhances robustness to packet direction and timing by using a TAM, providing a more robust representation of traffic than existing website fingerprinting defense strategies. RoFi outperforms existing WF attacks in open-world scenarios.
- **Trident** [18] trains a one-class classifier for each known class, enabling it to handle both sample and class increments. It facilitates fine-grained unknown traffic detection and model incremental updates.
- **RFG-HELAD** [33] combines a DNN with contrastive learning to create a K-classification model and employs a distance-based out-of-distribution detection algorithm for identifying unknown attacks.

[1]https://github.com/yungshenglu/USTC-TFC2016
[2]https://github.com/gcx-Yuan/Malicious_TLS

TABLE III
EVALUATION SCENARIOS

Scenarios	Known Classes(KC)	Unknown Classes(UC)
Scenario A-1	USTC-TFC2016\{U16}	U16
Scenario A-2	USTC-TFC2016\{U13, U15, U16}	U13, U15, U16
Scenario A-3	USTC-TFC2016\{U13, U15, U16, U18, U19}	U13, U15, U16, U18, U19
Scenario B-1	Malicious_TLS\{M3}	M3
Scenario B-2	Malicious_TLS\{M0, M1, M2}	M0, M1, M2
Scenario B-3	Malicious_TLS\{M0, M1, M2, M3, M4}	M0, M1, M2, M3, M4
Scenario C-1	Malicious_TLS	USTC-TFC2016
Scenario C-2	USTC-TFC2016	Malicious_TLS

Note: In Scenario C, we treat all the classes in one dataset as known classes and the samples in the other dataset as unknown classes.

TABLE IV
CLASSIFICATION PERFORMANCE COMPARISON AMONG FIVE CLOSED WORLD METHODS ON USTC-TFC2016 AND MALICIOUS_TLS DATASETS. RESULTS ARE IN THE FORMAT AVG.(±STD.) OBTAINED OVER 5-FOLD. THE BEST RESULTS ARE IN BOLDFACE

Task	**USTC-TFC2016**				**Malicious_TLS**			
Method	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
IIT	97.85(±0.23)	97.51(±0.52)	97.15(±0.21)	97.68(±0.34)	98.24(±0.55)	98.15(±0.25)	97.96(±0.48)	98.08(±0.98)
FastTraffic	96.93(±0.16)	96.57(±0.74)	94.99(±0.70)	95.53(±0.42)	91.52(±0.87)	90.89(±0.65)	90.99(±0.45)	91.12(±0.54)
GraphDApp	94.62(±0.72)	94.65(±0.82)	94.11(±0.52)	94.54(±0.63)	96.35(±0.65)	96.28(±0.48)	96.34(±0.24)	96.37(±0.49)
ACID	95.32(±0.61)	95.16(±0.29)	95.95(±0.51)	95.46(±0.39)	96.13(±0.21)	96.11(±0.15)	96.32(±0.98)	96.31(±0.19)
ET-BERT	99.13(±0.55)	99.22(±0.32)	99.12(±0.12)	99.16(±0.23)	98.89(±0.54)	98.95(±0.57)	98.84(±0.19)	98.97(±0.56)
RFG-HELAD	84.33(±0.51)	84.98(±0.26)	84.33(±0.15)	81.73(±0.56)	92.20(±0.12)	92.54(±0.23)	92.20(±0.15)	92.26(±0.54)
Open-Detect	**99.28**(±0.22)	**99.29**(±0.65)	**99.28**(±0.54)	**99.28**(±0.78)	**99.02**(±0.21)	**99.08**(±0.16)	**99.02**(±0.81)	**99.01**(±0.61)

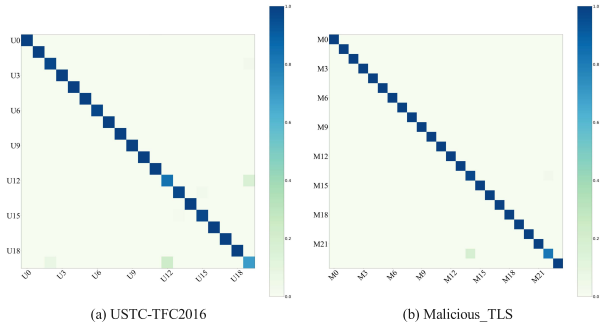
- **CVAE-EVT** [22] proposes a two-stage learning method that combines CVAE and EVT for fine-grained known/unknown intrusion detection.

B. Closed World performance(to RQ1)

The closed-world accuracy of a model is positively correlated with its open-world recognition capability. Improving the closed-world performance can, therefore, enhance the model's open-world ability [57]. To this end, we first evaluate the closed-world performance of Open-Detect.

Table IV reports the performance of Open-Detect alongside several baseline methods on two public datasets. From the Table, it is evident that Open-Detect achieves classification accuracies of 99.28% and 99.02% for the USTC-TFC2016 and Malicious_TLS datasets, respectively, outperforming all other methods. Among the baselines, ET-BERT achieves suboptimal results, with F1 scores of 99.16% and 98.87%, falling 0.1 percentage points short of Open-Detect. However, ET-BERT has a significantly larger number of parameters compared to Open-Detect, resulting in lower classification efficiency. While RFG-HELAD is an open-world model, its performance on closed sets is unsatisfactory. IIT, on the other hand, performs relatively well, leveraging the attention mechanism to fully capture intra- and inter-flow relationships.

Fig. 4 presents the confusion matrix of Open-Detect on two public datasets. As shown in Fig. 4, on the USTC-TFC2016 dataset, Open-Detect misclassified only a small number of samples as aliases, with more than half of the categories being ideally classified. On the Malicious_TLS dataset, Open-Detect misclassified 17.8% of M22 samples as M14, yet correctly classified the remaining 20 categories with 100% accuracy. These results demonstrate Open-Detect's outstanding closed-set performance, providing a solid foundation for its open-world capabilities.



(a) USTC-TFC2016 (b) Malicious_TLS

Fig. 4. Confusion matrices of Open-Detect on USTC-TFC2016 and Malicious_TLS datasets, where the vertical axis represents the true labels and the horizontal axis indicates the predicted labels.

C. Open World performance(to RQ2)

To assess the detection performance of our proposed Open-Detect model when dealing with unknown class traffic, we designed eight experimental scenarios using two datasets. Each scenario varied in the proportion of unknown classes, as detailed in Table III. For the first six scenarios, we randomly selected different numbers of traffic classes to create the training set, resulting in test sets with one to five unknown classes of traffic. Additionally, in Scenario C-1 and Scenario C-2, one dataset was used for training, while all classes from the other dataset were designated as unknown classes. This approach aimed to effectively evaluate and enhance the model's ability to address evolving and previously unseen data in real-world applications.

Overall, the model performs well across these scenarios, effectively distinguishing between normal and malicious

TABLE V
DETECTION PERFORMANCE COMPARISON OF THREE OPEN-WORLD METHODS IN SCENARIO A, WITH RESULTS PRESENTED AS AVG. (±STD.) OVER 5-FOLD. THE BEST RESULTS ARE BOLDED

Task	**Scenario A-1**		**Scenario A-2**		**Scenario A-3**	
Method	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
RoFi	87.84(±0.23)	86.45(±0.21)	86.48(±0.28)	85.97(±0.95)	83.89(±0.16)	84.45(±0.13)
Trident	65.00(±0.16)	64.34(±0.54)	48.20(±0.56)	44.62(±0.49)	43.01(±0.54)	40.34(±0.57)
RFG-HELAD	66.18(±0.57)	65.45(±0.62)	54.75(±0.26)	53.76(±0.13)	60.07(±0.22)	58.10(±0.64)
CVAE-EVT	89.82(±0.42)	88.23(±0.82)	87.18(±0.71)	86.92(±0.45)	82.45(±0.81)	82.56(±0.28)
Open-Detect	**98.20**(±0.65)	**98.24**(±0.51)	**89.22**(±0.67)	**89.10**(±0.35)	**95.51**(±0.35)	**91.17**(±0.45)

traffic. However, in Scenarios A and B, performance gradually declines as the number of unknown categories increases. The introduction of more unknown categories complicates the detection task, reducing the model's ability to classify all types of traffic accurately. At an FPR of 0.1, the TPR reaches its highest point, indicating that in some cases, a higher FPR is accepted to improve malicious traffic detection (i.e., increasing the TPR). This trade-off reflects a common challenge in security system design: balancing the risk of FPs against the cost of missed detections.

1) Performance on Scenario A: To comprehensively evaluate our proposed method, we benchmarked Open-Detect against various baseline methods and state-of-the-art models using the USTC-TFC2016 dataset. In this context, we examined three arrangements of known/unknown category splits: 19/1, 17/3, and 15/5.

Table V illustrates that Open-Detect outperforms all other methods, achieving detection accuracies of 98.20%, 89.22%, and 95.51% in the three scenarios, respectively. In contrast, CVAE-EVT, the closest competitor, yielded accuracies of 89.82%, 87.18%, and 82.45% across the same scenarios. While CVAE-EVT demonstrates competitiveness in certain aspects, a substantial performance gap remains relative to Open-Detect. Conversely, Trident and RFG-HELAD fall significantly short of expectations in these scenarios, with both accuracy and f1 score lagging considerably behind Open-Detect and RoFi.

As the number of unknown classes increases, Open-Detect's performance shows a slight decline. However, it continues to operate at a high efficacy level, exemplified by a 95.51% accuracy for the 15/5 Scenario. This underscores the model's robust detection capabilities, even when confronted with a greater variety of unknown traffic. In contrast, models like CVAE-EVT, Trident, and RFG-HELAD exhibit markedly poorer performance as the number of unknown categories rises. The increasing presence of unknown categories complicates the feature space, introducing greater intricacy and variability. Each newly introduced unknown category presents unique feature patterns, some of which may resemble known traffic, thereby blurring decision boundaries and complicating classification tasks. Open-Detect effectively mitigates these challenges by maintaining intra-class compactness and ensuring clear inter-class separation properties, enabling it to consistently outperform competitors.

2) Performance on Scenario B: We evaluated Open-Detect against various baselines and state-of-the-art models using the Malicious_TLS dataset, testing three known/unknown category splits: 23/1, 21/3, and 19/5.

As shown in Table VI, Open-Detect achieved remarkable detection accuracies of 90.20%, 90.34%, and 85.94% in Scenarios B-1, B-2, and B-3, respectively, along with f1 scores of 90.71%, 89.62%, and 85.97%. These results are notably superior to those of other methods, demonstrating Open-Detect's robust detection capability and balanced performance. Even in Scenario B-3, where the number of unknown categories increases, Open-Detect maintains its accuracy and f1 score at 85.94% and 85.97%, respectively, showcasing its robustness in managing complex data distributions.

While CVAE-EVT performs sub-optimally across all scenarios, it consistently falls short of Open-Detect's results. Trident and RFG-HELAD exhibit even more pronounced performance degradation, especially as the number of unknown categories grows. This is particularly evident in Scenario B-3, where their accuracy and f1 scores are substantially lower than those of Open-Detect and CVAE-EVT.

3) Performance on Scenario C: Finally, we compared Open-Detect with several baseline methods and state-of-the-art models across two datasets, USTC-TFC2016 and Malicious_TLS, analyzing two known/unknown category splits: 24/20 and 20/24. The results of the detection performance comparison in Scenarios C-1 and C-2 are detailed in Table VII.

In Scenario C-1, Open-Detect exhibits remarkable performance with an accuracy of 93.31% and an f1 score of 93.56%, significantly outperforming competing methods. Notably, Open-Detect's accuracy and f1 score are approximately 4.72% and 6.33% higher than those of RFG-HELAD, respectively. In Scenario C-2, Open-Detect's performance is even more exceptional, achieving an accuracy of 97.45% and an f1 score of 98.65%, figures that far exceed those achieved by other methods. Compared to RFG-HELAD, Open-Detect's accuracy and f1 score are approximately 16.62% and 20.46% higher, respectively.

The CVAE-EVT model performs exceptionally well in Scenario C-2, achieving an accuracy of 91.52% and an f1 score of 93.14%, ranking second only to Open-Detect. Meanwhile, RoFi maintains relatively stable performance across both scenarios, although there remains a notable gap when compared to the superior results of Open-Detect. Trident, on the other hand, exhibits more volatility in its performance. In Scenario C-1, its accuracy and f1 score are 78.51% and 77.51%, respectively, with a slight improvement in Scenario C-2 to 81.56% and 85.31%. This inconsistency is attributed to Trident's exclusive

TABLE VI

DETECTION PERFORMANCE COMPARISON OF THREE OPEN-WORLD METHODS IN SCENARIO B, WITH RESULTS PRESENTED AS AVG. (±STD.) OVER 5-FOLD. THE BEST RESULTS ARE BOLDED

Task	**Scenario B-1**		**Scenario B-2**		**Scenario B-3**	
Method	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
RoFi	85.46(±0.33)	84.56(±0.45)	83.88(±0.26)	83.49(±0.82)	80.65(±0.21)	81.51(±0.74)
Trident	73.07(±0.54)	75.26(±0.51)	73.43(±0.82)	73.94(±0.55)	64.72(±0.87)	62.16(±0.84)
RFG-HELAD	74.27(±0.23)	76.93(±0.22)	78.72(±0.42)	80.39(±0.47)	63.13(±0.12)	65.31(±0.18)
CVAE-EVT	86.12(±0.58)	86.38(±0.46)	85.21(±0.45)	86.01(±0.66)	81.48(±0.73)	81.83(±0.69)
Open-Detect	**90.20**(±0.19)	**90.71**(±0.15)	**90.34**(±0.24)	**89.62**(±0.21)	**85.94**(±0.31)	**85.97**(±0.29)

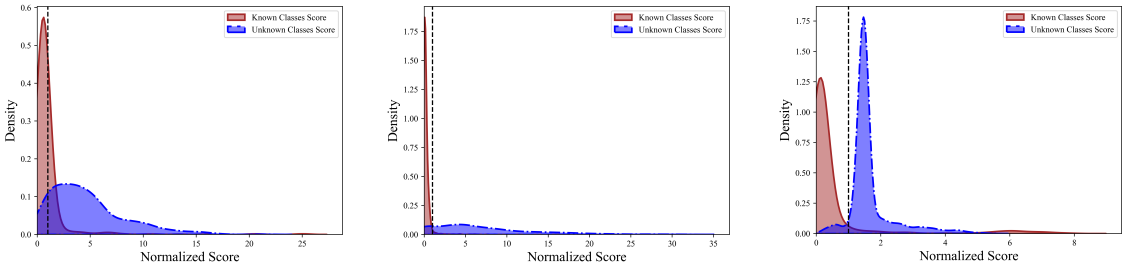


Fig. 5. Effect of different thresholds on Open-Detect.

TABLE VII

DETECTION PERFORMANCE COMPARISON OF THREE OPEN-WORLD METHODS IN SCENARIO C, WITH RESULTS PRESENTED AS AVG. (±STD.) OVER 5-FOLD. THE BEST RESULTS ARE BOLDED

Task	**Scenario C-1**		**Scenario C-2**	
Method	Accuracy	F1-Score	Accuracy	F1-Score
RoFi	83.56(±0.41)	84.55(±0.32)	84.52(±0.34)	85.12(±0.32)
Trident	78.51(±0.21)	77.51(±0.12)	81.56(±0.35)	85.31(±0.43)
RFG-HELAD	88.59(±0.44)	87.23(±0.56)	80.83(±0.54)	78.19(±0.48)
CVAE-EVT	91.46(±0.23)	91.62(±0.44)	91.52(±0.83)	93.14(±0.62)
Open-Detect	**93.31**(±0.34)	**93.56**(±0.38)	**97.45**(±0.29)	**98.65**(±0.17)

TABLE VIII

AVERAGE TESTING TIMES OF DIFFERENT METHODS

Method	**FET** (ms)	**CPT** (ms)	**Total time** (ms)
RoFi	0.11	0.52	0.63
Trident	0.15	1.03	1.18
RFG-HELAD	0.16	0.41	0.57
CVAE-EVT	0.16	0.66	0.82
Open-Detect	0.20	0.93	1.13

reliance on reconstruction error for detecting unknown traffic, which limits its capacity to effectively manage complex and diverse unknown categories.

We divide the complexity of the model into two aspects: feature extraction time and classifier prediction time. FET stands for Feature Extraction Time, which refers to the time required for converting the raw traffic into a grayscale image. The prediction time refers to the time spent in classifying each flow, while the total time is the sum of feature extraction time and classifier prediction time. In the Open-Detect model, the feature extraction time is the time required to convert the raw traffic into a grayscale image.

Table VIII presents the feature extraction time and the average detection time per network flow for different methods. Open-Detect requires an average of 1.13ms per network flow, which is better than Trident's 1.18ms and slightly higher than the 0.82ms of the CVAE-EVT model. RFG-HELAD, using a DNN model, achieves the fastest speed. Trident maintains multiple single-class one-class learners, which results in slightly lower detection efficiency. Overall, Open-Detect does not have a significant disadvantage in detection efficiency compared to the current methods.

D. Results Visualization (to RQ3)

1) Visualizing Threshold Effects: Fig. 5 demonstrates the impact of varying thresholds on the Open-Detect model across three scenarios: A-2, B-2, and C-1. In each subplot, the horizontal axis represents the normalized scores of the samples. The red curve corresponds to the scores of known categories, and the blue curve pertains to unknown categories. The vertical axis indicates the proportion of samples corresponding to these scores. The Normalized Score is calculated as $dist/threshold$, where $dist$ is the distance of a test sample from the nearest Gaussian prototype in the latent space. The black dashed line represents the threshold at which test samples are classified as unknown traffic. If the distance $dist$ of a test sample exceeds the set threshold, it is classified as unknown traffic.

In all three scenarios, it is evident that most distances, $dist$, for known categories fall below the set threshold, while many scores for unknown categories surpass this threshold. This underscores the model's effectiveness in distinguishing between known and unknown categories, driven by the pronounced score differences. Open-Detect successfully achieves this differentiation by maintaining intra-class compactness and

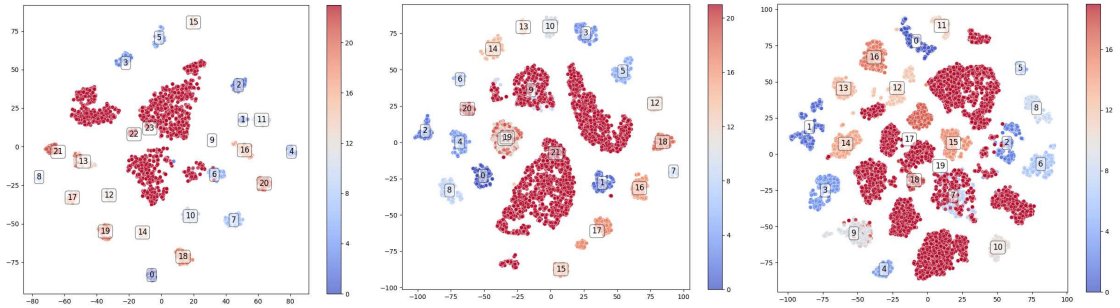


Fig. 6. Feature visualization of known and unknown classes on the Malicious_TLS dataset, where the unknown class is represented by red dots in each subplot. In Scenario B-1, the numbers 0 to 22 represent known classes, and 23 represents the unknown class M3. In Scenario B-2, 21 represents the unknown classes M0, M1, and M2. In Scenario B-3, 19 represents the unknown classes M0, M1, M2, M3, and M4.

TABLE IX

THE ABLATION STUDY OF OPEN-DETECT IN SCENARIO B, WHERE ACCURACY AND F1-SCORE REPRESENT THE DETECTION METRICS FOR UNKNOWN MALICIOUS TRAFFIC

Method	Scenario B-1			Scenario B-2			Scenario B-3		
	Accuracy	F1-Score	AUROC	Accuracy	F1-Score	AUROC	Accuracy	F1-Score	AUROC
Open-Detect (default)	90.20	90.71	89.69	90.34	89.10	93.17	85.94	81.51	85.96
w/o Discriminative constraints	65.27	67.13	67.84	62.81	–	62.15	64.91	63.85	68.05
w/o Generative constraints	56.83	69.40	–	63.21	70.47	61.85	56.53	–	–
w/o Header	69.11	73.68	70.56	50.28	66.61	–	54.51	65.31	–
w/o Payload	61.41	54.54	65.21	63.78	68.51	65.03	53.01	66.42	–
w/o IP Masking	71.82	75.94	77.06	51.20	66.52	–	56.49	65.97	55.82
w/o Ciphertext	85.60	86.56	91.86	75.86	76.78	73.83	83.29	83.65	88.11

Note: – means that the value is lower than 50%, which is nearly the result of random guessing.

ensuring inter-class separation. This capability is evident not only in scenarios with fewer known and unknown categories but also in scenarios with an increased number of unknown categories, where the model continues to deliver high detection performance.

Overall, the Open-Detect model effectively utilizes the distance *dist* as a discriminative criterion, achieving clear separation between known and unknown categories by selecting appropriate thresholds.

2) Feature Space Analysis: Fig. 6 illustrates the feature distribution of test samples in the latent space across three scenarios (Scenario B-1 to Scenario B-3) using the Malicious_TLS dataset. In these visualizations, the darkest red scatter points represent the unknown categories.

In Scenario B-1, the unknown class consists of a single category, with classes 22 and 6 showing minimal overlap with the unknown class. As the number of unknown categories increases, Scenario B-2 includes three classes within the unknown category, and classes 9 and 6 exhibit slight overlap with the unknown category. By Scenario B-3, when the number of unknown categories increases to five, the overlap between known and unknown categories expands further, with classes 7, 9, 17, and 18 showing more substantial overlap with the unknown category.

This progression indicates that as the number of unknown categories grows, the difficulty for Open-Detect in distinguishing between known and unknown categories increases, which could lead to more frequent misclassification. Nevertheless, despite this challenge, Open-Detect continues to maintain a relatively strong ability to differentiate between categories, particularly among known categories that show less overlap with unknown ones.

E. Ablation Experiment

To further validate the design of Open-Detect, we conducted an ablation study to evaluate the contribution of each component in Open-Detect. The corresponding results are shown in Table IX.

1) Model-Level Ablation: The discriminative constraint, by maximizing the inter-class distance, enforces separation of known classes. However, the absence of a generative constraint for intra-class compression causes the distribution range of samples from the same class to expand in the feature space. Despite this, the discriminative constraint still maintains inter-class separation, ensuring that the Open-Detect model retains a certain level of detection capability. The detection accuracy of Open-Detect is above 56.53%.

If Open-Detect only has the generative constraint, the generative constraint increases feature compactness by compressing the intra-class distribution. However, without the discriminative constraint for inter-class separation, known classes overlap in the feature space, making it impossible to distinguish between different classes. This results in a significant degradation in model performance. In Scenarios B-1 to B-3, the AUROC value is below 68.05%.

2) Data-Level Ablation: When all header bytes in the data packet are omitted, the detection performance of unknown malicious traffic significantly decreases. In all three scenarios, the detection accuracy drops by 21.09% to 40.06%. This indicates that key fields in the header play a decisive role in detecting unknown malicious traffic.

TABLE X
THE FIELDS CORRESPONDING TO THE FOUR MOST IMPORTANT PIXELS

Pixel Position	Hex	Field Name	Technical Meaning
475	0x01	Version	TLS 1.0 Protocol Version
473	0xd2	Payload Length	210 bytes of Client Hello data
471	0x00	Payload Length	210 bytes of Client Hello data
469	0xd6	TLS Layer Length	214 bytes of TLS layer data

Regarding the data packet payload, the ablation results show that from Scenario B-1 to Scenario B-3, the detection accuracy drops by 26.56% to 32.93%. For encrypted TLS traffic, both the header and the payload contain a large amount of key information.

When the IP address is not anonymized, there is still a noticeable decline in detection accuracy. This phenomenon validates the necessity of anonymizing the original traffic: identification information, such as IP addresses, introduces domain-dependent bias during model training.

After replacing all encrypted content in the payload with 0×00, we find that the detection performance of unknown malicious traffic still decreases to varying extents. This suggests that the encrypted payload still contains some key information and cannot be completely discarded. This also validates the effectiveness of directly constructing grayscale images from the payload (including both plaintext and ciphertext) as proposed in this paper.

F. Explainability

To make Open-Detect more interpretable, we use the gradient method to compute the gradients of the Open-Detect model with respect to the input pixels. The gradient method is a differential-based model explanation technique, and its core principle is to quantify the importance of pixels, or fields in a data packet, by calculating the partial derivative of the model's output with respect to the input pixels. Given an input grayscale image $x \in \mathbb{R}^{32\times32}$ and a model f, for a target class c, the importance of pixel (i, j) is calculated as:

$$I_{i,j} = \left| \frac{\partial f_c(x)}{\partial x_{i,j}} \right|,$$

where $f_c(x)$ represents the model's output score for class c. The mathematical interpretation of this formula is:

- The partial derivative $\frac{\partial f_c}{\partial x_{i,j}}$ indicates the rate of change of the model's output due to a unit change in pixel $x_{i,j}$.
- The absolute value ensures that both positive and negative gradients are equally important (increasing or decreasing a pixel value can both affect the prediction).

Finally, the importance map is normalized:

$$\hat{I}_{i,j} = \frac{I_{i,j} - \min(I)}{\max(I) - \min(I)}$$

to obtain an importance map in the range [0,1]. If a small perturbation to a pixel significantly alters the model's output, that pixel is considered crucial for the decision. This approach is used to calculate the most important feature fields in a data packet.

The following is an explanation using Arachni malware traffic as an example:

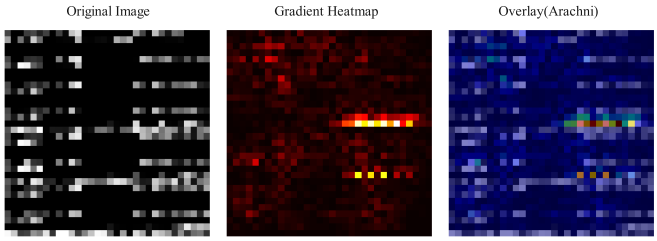


Fig. 7. The original image, gradient heatmap, and overlay image of Arachni traffic.

As shown in Fig. 7, the original image is a 32×32 grayscale image input into the model. In the Gradient Heatmap, the red/yellow pixels represent high-importance areas (gradient values close to 1), while the blue/black pixels represent low-importance areas (gradient values close to 0). The overlay image superimposes a semi-transparent red heatmap on the original grayscale image background, intuitively showing the key field locations. By analyzing the Gradient Heatmap, we identify the four most important pixel positions (sorted by rows) as 475, 473, 471, and 469. As shown in Table X, these positions correspond to the key fields in the fourth packet (Client Hello).

These feature combinations form the fingerprint of arachni traffic, which is the key basis for the Open-Detect model to accurately identify such threats.

VII. DISCUSSION AND LIMITATIONS

A. Feature Extraction

The grayscale image representation inherently fails to capture the temporal dynamic features of network flows. This limitation arises from its implementation of statically aggregating packet byte sequences into 32×32 images, which does not include inter-packet timing interval information.

B. Limitations

The model training requires substantial GPU memory, which limits its efficient deployment on low-resource devices. However, we can effectively reduce the model's GPU memory usage through model quantization, without compromising its performance.

C. Future Works

Our work did not consider adversarial evaluation of advanced obfuscation techniques such as TLS 1.3, VPN, and Tor traffic. In an open network environment, these three types of traffic pose significant challenges to the detection of unknown malicious traffic. Fully encrypted traffic can lead to the loss of key bytes, causing detection methods based on raw bytes to potentially fail. In future work, we will explore detection methods for unknown malicious traffic targeting these advanced obfuscation techniques.

VIII. CONCLUSION

In this study, we proposed a novel model for detecting unknown malicious traffic, called Open-Detect, which is built upon a comprehensive theoretical framework and utilizes CVAE to classify known traffic classes and detect unknown

traffic classes. Open-Detect is a generative model that, compared to existing generative models, relies on generative constraints to achieve intra-class compactness while using discriminative constraints to achieve inter-class separation. This approach effectively minimizes the risk of misclassification for both known and unknown classes. Experimental results on real-world datasets demonstrate that Open-Detect outperforms all current closed-world and open-world methods, achieving superior performance in both known class classification and unknown class detection.

REFERENCES

[1] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.

[2] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.

[3] Q. Yuan et al., "Multi-agent for network security monitoring and warning: A generative AI solution," *IEEE Netw.*, vol. 39, no. 5, pp. 114–121, Sep. 2025.

[4] X. Wang et al., "Entropy-regulated cross-modal generative fusion for multimodal network intrusion detection," *Inf. Fusion*, vol. 126, Feb. 2026, Art. no. 103581.

[5] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1156–1167, 2024.

[6] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.

[7] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 589–606.

[8] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[9] K. Wang et al., "BARS: Local robustness certification for deep learning based traffic analysis systems," in *Proc. NDSS*, 2023, doi: 10.14722/ndss.2023.24508.

[10] T. Lu and J. Wang, "DOMR: Toward deep open-world malware recognition," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1455–1468, 2024.

[11] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, "Toward open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 7, pp. 1757–1772, Jul. 2013.

[12] C. Geng, S.-J. Huang, and S. Chen, "Recent advances in open set recognition: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3614–3631, Oct. 2021.

[13] R. Chen, L. Luo, X. Wang, B. Ren, D. Guo, and S. Zhu, "Knowing the unknowns: Network traffic detection with open-set semi-supervised learning," *Comput. Netw.*, vol. 251, Sep. 2024, Art. no. 110630.

[14] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.

[15] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 397–405.

[16] Q. Meng et al., "Beyond known threats: A novel strategy for isolating and detecting unknown malicious traffic," *J. Inf. Secur. Appl.*, vol. 89, Mar. 2025, Art. no. 103920.

[17] Q. Yuan, G. Gou, Y. Zhu, Y. Zhu, G. Xiong, and Y. Wang, "MCRe: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 133–147, 2024.

[18] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, May 2024, pp. 1608–1619.

[19] H. He, Y. Lai, Y. Wang, S. Le, and Z. Zhao, "A data skew-based unknown traffic classification approach for TLS applications," *Future Gener. Comput. Syst.*, vol. 138, pp. 1–12, Jan. 2023.

[20] Y. Gu, Y. Lai, and Y. Wang, "Zen-tor: A zero knowledge known-unknown traffic classification method," in *Proc. GLOBECOM - IEEE Global Commun. Conf.*, Dec. 2022, pp. 885–890.

[21] X. Li, B. Feng, T. Zang, X. Xu, S. Zhao, and J. Ma, "Facing unknown: Open-world encrypted traffic classification based on contrastive pre-training," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2023, pp. 1255–1260.

[22] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021.

[23] C. Sheng, Y. Yao, W. Li, W. Yang, and Y. Liu, "Unknown attack traffic classification in SCADA network using heuristic clustering technique," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2625–2638, Sep. 2023.

[24] Y. Qing et al., "Low-quality training data only? A robust framework for detecting encrypted malicious network traffic," 2023, *arXiv:2309.04798*.

[25] J. Ma, Q. Chai, J. Liu, Q. Yin, P. Wang, and Q. Zheng, "XTQA: Span-level explanations for textbook question answering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 16493–16503, Nov. 2024.

[26] J. Ma et al., "Robust visual question answering: Datasets, methods, and future challenges," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5575–5594, Aug. 2024.

[27] Y. Zhao, C. Xia, T. Wang, M. Liu, and Y. Li, "Few-shot encrypted malicious traffic classification via hierarchical semantics and adaptive prototype learning," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2024, pp. 399–409.

[28] W. Cai, C. Hou, M. Cui, B. Wang, G. Xiong, and G. Gou, "Incremental encrypted traffic classification via contrastive prototype networks," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110591.

[29] Z. Liu, L. Cai, L. Zhao, A. Yu, and D. Meng, "Towards open world traffic classification," in *Proc. Int. Conf. Inf. Commun. Secur.*, Chongqing, China, Nov. 2021, pp. 331–347.

[30] T. Dahanayaka, Y. Ginige, Y. Huang, G. Jourjon, and S. Seneviratne, "Robust open-set classification for encrypted traffic fingerprinting," *Comput. Netw.*, vol. 236, Nov. 2023, Art. no. 109991.

[31] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.

[32] Z. Gao, J. Li, L. Wang, Y. He, and P. Yuan, "CM-UTC: A cost-sensitive matrix based method for unknown encrypted traffic classification," *Comput. J.*, vol. 67, no. 7, pp. 2441–2452, Jul. 2024.

[33] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "RFG-HELAD: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5895–5910, 2024.

[34] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, Jun. 2014.

[35] Y. Chen, Z. Li, J. Shi, G. Gou, C. Liu, and G. Xiong, "Not afraid of the unseen: A Siamese network based scheme for unknown traffic discovery," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2020, pp. 1–7.

[36] L. Yang, A. Finamore, F. Jun, and D. Rossi, "Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4103–4118, Dec. 2021.

[37] E. Nalisnick, A. Matsukawa, Y. Whye Teh, D. Gorur, and B. Lakshminarayanan, "Do deep generative models know what they don't know?," 2018, *arXiv:1810.09136*.

[38] S. Le, Y. Lai, Y. Wang, and H. He, "An adaptive classification and updating method for unknown network traffic in open environments," *Comput. Netw.*, vol. 238, Jan. 2024, Art. no. 110114.

[39] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–11.

[40] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[41] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[42] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.

[43] P. Sun, X. Yun, S. Li, T. Yin, C. Si, and J. Xie, "AdvTG: An adversarial traffic generation framework to deceive DL-based malicious traffic detection models," in *Proc. ACM Web Conf.*, Apr. 2025, pp. 3147–3159.
[44] E. Ahmadzadeh, H. Kim, O. Jeong, N. Kim, and I. Moon, "A deep bidirectional LSTM-GRU network model for automated ciphertext classification," *IEEE Access*, vol. 10, pp. 3228–3237, 2022.
[45] R. Xie, X. Chen, X. Zhang, and G. Shi, "Block cipher algorithm identification based on CNN-transformer fusion model," in *Proc. Chin. Conf. Pattern Recognit. Comput. Vis. (PRCV)*, 2024, pp. 97–110.
[46] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
[47] H. Hu and K. Yuan, "Identification of cryptographic algorithms based on CNN," in *Proc. 4th Int. Conf. Comput., Artif. Intell. Control Eng.*, Jan. 2025, pp. 182–186.
[48] J. Liu, J. Tian, W. Han, Z. Qin, Y. Fan, and J. Shao, "Learning multiple Gaussian prototypes for open-set recognition," *Inf. Sci.*, vol. 626, pp. 738–753, May 2023.
[49] C. Doersch, "Tutorial on variational autoencoders," 2016, *arXiv:1606.05908*.
[50] Mathematics Stack Exchange. (2014). *Normal Distribution Tail Probability Inequality*. [Online]. Available: https://math.stackexchange.com/users/6179/did, https://math.stackexchange.com/q/988835
[51] E. Seneta, "On the history of the strong law of large numbers and Boole's inequality," *Historia Mathematica*, vol. 19, no. 1, pp. 24–39, Feb. 1992.
[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[53] Q. Meng et al., "IIT: Accurate decentralized application identification through mining Intra- and inter-flow relationships," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 1, pp. 394–408, Feb. 2025.
[54] Y. Xu, J. Cao, K. Song, Q. Xiang, and G. Cheng, "FastTraffic: A lightweight method for encrypted traffic fast classification," *Comput. Netw.*, vol. 235, Nov. 2023, Art. no. 109965.
[55] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
[56] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.
[57] S. Vaze, K. Han, A. Vedaldi, and A. Zisserman, "Open-set recognition: A good closed-set classifier is all you need?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

Qianwei Meng is currently pursuing the Ph.D. degree with Henan Key Laboratory of Network Cryptography Technology, Information Engineering University. His research interests include network security and unknown traffic detection.

Jing Tao received the B.S. and M.S. degrees in automatic control from Xi'an Jiaotong University, Xi'an, China, in 2001 and 2006, respectively. He is currently pursuing the Ph.D. degree with the Systems Engineering Institute. He is a Teacher with Xi'an Jiaotong University. His research interests include internet traffic measurement and modeling, traffic classification, abnormal detection, and botnet.

Qingjun Yuan is currently a Post-Doctoral Researcher with the MoE Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University. He is also a Lecturer with Henan Key Laboratory of Network Cryptography Technology, Information Engineering University. His research interests include side-channel analysis and encrypted traffic analytics.

Guangsong Li received the Ph.D. degree from Henan Key Laboratory of Network Cryptography Technology. He is currently a Professor with Henan Key Laboratory of Network Cryptography Technology. His research interests include blockchain security, security protocol, information security, and formal methods.

Yongjuan Wang received the Ph.D. degree from Information Engineering University in 2009. She is currently working with Henan Key Laboratory of Network Cryptography Technology. Her main research interests include cryptographic analysis and cyberspace security.

Bing Gao received the master's degree from Chongqing University. He is currently working with Henan Key Laboratory of Network Cryptography Technology. His research interests include combinatorial mathematics and encrypted traffic analysis.

Siqi Lu received the Ph.D. degree from Henan Key Laboratory of Network Cryptography Technology. He is currently a Lecturer with Henan Key Laboratory of Network Cryptography Technology. His research interests include blockchain security, formal methods, security protocol, and big data security.